

第4章 函数与方法

建议学时:4-6 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解"函数是一等公民":函数可赋值、传参、返回,远超 C 的函数指针
- 掌握参数传递语义:值传递与"对象按引用共享"的区别,函数内修改是否影响外部
- 彻底掌握 this 的四种绑定规则(默认/隐式/显式/new)与箭头函数的特殊规则
- 熟练使用 call/apply/bind 并理解三者差异
- 理解闭包的机制与生产用途,以及它与 C 函数指针的本质区别
- 掌握高阶函数 map/filter/reduce 与回调风格
- 理解"C 中怎么做→JS 中怎么做":main 入口、参数与返回的差异

💡 从 C 到 JavaScript:本章主题如何迁移

C 里函数是"代码段":有地址,有调用约定,有函数指针可以传来传去。JS 里函数更进一步,是 **第一等公民**:函数就是对象,可以存进数组、当参数传给另一个函数、被另一个函数返回、还能"记住"定义时的环境——后者正是闭包,是 C 的函数指针做梦都做不到的能力。

C 的函数指针用 `int (*cmp)(const void*, const void*)` 给 `qsort` 传比较器;JS 用 `arr.sort((a, b) => a - b)`,语法简洁一个数量级,而且回调能捕获外层变量(闭包),C 只能靠额外参数传递上下文。

C 里没有 this:结构体成员函数用 `obj.method(&obj, ...)` 显式传自己。JS 的 this 是"调用时决定"的隐式参数,灵活但反直觉——四种绑定规则必须逐个记牢,这是本章最重要的部分。

参数传递也完全不同:C 全部传值,想改外部就传指针;JS 里原始类型传值(改不动外部),对象传"引用的副本"(能改到外部对象的属性,但重新赋值不影响外部)。这个"对象共享"语义是生产 bug 高发区,必须建立肌肉记忆。

最后,JS 没有 C 那样的 main 入口:Node 程序从模块顶层开始执行,浏览器从事件驱动开始。但模块可以导出一个 `main()` 函数并调用它,风格上向 C 靠拢,可读性更好。

📦 前置知识自查

- C 的函数定义、声明与调用,形参实参、return
- C 的传值与传地址(`int* p`),修改外部变量的两种姿势
- C 的函数指针:typedef 函数指针类型、回调函数
- C 的作用域:局部变量在函数退出后销毁
- C 的 static 局部变量(生命周期延长)——JS 闭包与之形似神不同

本章学习路线

1. **第一阶段(语法与传参)**:读 §4.1–§4.3,把所有示例在 REPL 跑通
2. **第二阶段(this,本章核心)**:读 §4.4–§4.6,自己写"四种绑定"的最小例子,务必亲手验证
3. **第三阶段(闭包与高阶函数)**:读 §4.7–§4.8,理解闭包"捕获环境"的机制
4. **第四阶段(实验)**:完成章末"可配置统计管线"
5. **验收标准**:不看资料,能说出四种 this 绑定场景与箭头函数的例外,能写出一个闭包计数器

本章知识导图

- 第4章 函数与方法
 - §4.1 定义与调用:声明式/表达式/箭头函数
 - §4.2 参数传递:值传递与对象共享、默认参数、rest、arguments
 - §4.3 返回值与多返回值(解构)
 - §4.4 this 绑定四规则:默认/隐式/显式/new
 - §4.5 call/apply/bind 详解
 - §4.6 箭头函数:this/arguments 的词法绑定
 - §4.7 闭包:捕获环境、计数器/工厂/私有状态
 - §4.8 高阶函数:map/filter/reduce、回调
 - §4.9 递归、入口与组织

§4.1 定义与调用:三种写法

```
// 写法一:函数声明(可提升,与 C 的函数声明类似)
function add(a, b) {
  return a + b;
}
console.log(add(1, 2));           // 3

// 写法二:函数表达式(不可提升,必须定义后使用)
const sub = function (a, b) {
  return a - b;
};
console.log(sub(5, 2));           // 3

// 写法三:箭头函数(ES2015,无 this/arguments,见 §4.6)
const mul = (a, b) => a * b;
console.log(mul(3, 4));           // 12

// 函数是对象:可以当属性、存数组、传参
const ops = [add, sub, mul];
console.log(ops[2](2, 5));        // 10,从数组里取函数调用
console.log(typeof add);          // 'function'
console.log(add.name);            // 'add',函数自带名字属性
```

⚠ 常见误区 4.1:函数声明会被提升,函数表达式不会

与 C 的"函数必须先声明后使用"不同,JS 的函数声明会在整个作用域顶部生效,先调用后声明也合法;但 `const` 声明的函数表达式严格遵守 TDZ,定义前调用抛 `TypeError`。生产规范:一律用 `const` + 函数表达式或箭头函数,行为可预测,也避免命名冲突。

§4.2 参数传递:值传递 + 对象共享

💡 直觉先行:传参永远"传副本",但副本可能是引用

C 里传结构体是整块拷贝,传指针是拷贝指针本身;JS 里只有一种传参方式:传值的副本。区别在于:原始类型 (`number/string/bool`)的副本是值本身,函数内修改不影响外部;对象(含数组、函数)的副本是"引用",引用指向同一个对象,函数内修改属性会反映到外部,但给参数重新赋值不会影响外部变量。

场景	C	JS
改外部 int	传 <code>int*</code> 再 <code>*p = x</code>	传对象包一层,或函数返回新值
改结构体成员	传 <code>struct*</code>	传对象,直接改属性即可
整个数组传参	退化为指针,配合长度参数	传引用,自带 <code>length</code>
形参重新赋值	不影响实参	不影响实参(引用副本)

```

// 原始类型:改不动外部
function tryChange(n) {
  n = 999;
}
let x = 1;
tryChange(x);
console.log(x);           // 1 ← 不受影响

// 对象:改属性影响外部
function addScore(student, score) {
  student.scores.push(score); // 通过引用改到了外部对象
}
const s = { name: '李雷', scores: [80] };
addScore(s, 95);
console.log(s.scores);    // [ 80, 95 ] ← 外部被改了

// 但重新赋值参数不影响外部
function rebind(obj) {
  obj = { name: '全新对象' }; // 只是让形参指向新对象
}
const o = { name: '原对象' };
rebind(o);
console.log(o.name);      // 原对象 ← 外部不受影响

// C 的"输出参数"写法在 JS 里改用什么?→ 返回新值
function pushClone(arr, v) {
  return [...arr, v];      // 返回新数组(不可变风格)
}
const nums = [1, 2];
const nums2 = pushClone(nums, 3);
console.log(nums, nums2);  // [1,2] [1,2,3]

```

4.2.1 默认参数与 rest 参数

```
// 默认参数:C 没有(需手动判断);JS 在形参表直接给
function greet(name = '匿名', level = '同学') {
  return `你好,${level} ${name}`;
}
console.log(greet());           // 你好,同学 匿名
console.log(greet('李雷'));     // 你好,同学 李雷
console.log(greet('韩梅梅', '老师')); // 你好,老师 韩梅梅
// 注意:传 undefined 触发默认值,传 null 不触发
console.log(greet(undefined, null)); // 你好,null 匿名

// rest 参数:收集多余参数为数组(类似 C 的变参,但类型安全)
function sum(...nums) {
  let total = 0;
  for (const n of nums) total += n;
  return total;
}
console.log(sum(1, 2, 3, 4));    // 10
console.log(sum());              // 0

// arguments:老式变参对象(箭头函数没有!)
function oldSum() {
  let total = 0;
  for (let i = 0; i < arguments.length; i++) {
    total += arguments[i];
  }
  return total;
}
console.log(oldSum(1, 2, 3));     // 6,生产推荐用 ...rest 替代 arguments
```

工程建议 4.1:参数超过 3 个就用"配置对象"

C 里参数一多就用结构体;JS 对应物是配置对象 + 解构默认值,调用方可读性极高:

```
function createServer({ host = '0.0.0.0', port = 3000, timeout = 5000, logger = console } = {}) {
  return { host, port, timeout, logger };
}
const srv = createServer({ port: 8080 });
console.log(srv.port);           // 8080,其余取默认
```

§4.3 返回值:return 与解构接收

```
// C 一次只能返回一个值(结构体除外);JS 用对象/数组解构"模拟多返回值"
function analyze(nums) {
  const sum = nums.reduce((a, b) => a + b, 0);
  const max = Math.max(...nums);
  return { sum, max, count: nums.length }; // 返回对象
}
const { sum, max, count } = analyze([3, 9, 5]);
console.log(sum, max, count); // 17 9 3

// 没有 return 的函数返回 undefined
function noop() {}
console.log(noop()); // undefined
```

§4.4 this 绑定四规则:本章核心

💡 直觉先行:this 是"调用时"决定的隐式参数

C 的结构体函数写法 `obj.method(&obj)` 把"自己"作为第一个参数显式传入;JS 把这个参数藏了起来,叫 `this`,由调用方式 决定它指向谁。同一函数,四种调用方式,this 四个值:

```

function who() {
  return this === undefined ? 'undefined(严格模式)' : String(this);
}

// 规则一:默认绑定—独立调用,this 为 undefined(严格模式)或全局对象(非严格)
'use strict';
function show() { console.log(this); }
show(); // undefined ← 默认绑定,严格模式下

// 规则二:隐式绑定—作为对象方法调用,this 指向该对象
const user = {
  name: '李雷',
  hello() { return `我是 ${this.name}`; },
};
console.log(user.hello()); // 我是 李雷 ← this 是 user

// 隐式绑定陷阱:方法被"拿出来"单独调用,this 丢失!
const fn = user.hello;
console.log(fn()); // TypeError: Cannot read properties of undefined
// 原因:fn() 是独立调用,this 变 undefined,this.name 崩溃

// 规则三:显式绑定—call/apply/bind 指定 this
const other = { name: '韩梅梅' };
console.log(user.hello.call(other)); // 我是 韩梅梅
console.log(user.hello.apply(other)); // 我是 韩梅梅
const bound = user.hello.bind(other);
console.log(bound()); // 我是 韩梅梅 ← bind 返回新函数

// 规则四:new 绑定—作为构造函数调用,this 指向新对象
function Person(name) {
  this.name = name; // this 是 new 创建的新对象
}
const p = new Person('王芳');
console.log(p.name); // 王芳

```

★ 必须背下的四规则优先级

new 绑定 > 显式绑定(call/apply/bind) > 隐式绑定(对象方法) > 默认绑定(独立调用)。判断 this 时先看调用形式:前面有没有 new?有没有 call/apply/bind?有没有"."(对象方法)?都没有就是默认绑定。箭头函数不参与以上任何规则,见 §4.6。

§4.5 call / apply / bind:显式绑定的三兄弟

```
function introduce(city, year) {
  return `${this.name},${city},${year}`;
}
const person = { name: '李雷' };

// call:逐个传参
console.log(introduce.call(person, '北京', 2026));
// apply:参数用数组传(与 call 等价,生产已少用)
console.log(introduce.apply(person, ['北京', 2026]));
// bind:绑定 this 与部分参数,返回新函数
const intro = introduce.bind(person, '上海');
console.log(intro(2026));           // 李雷,上海,2026 ← 参数被"预填"

// 经典用途:借用方法(数组方法借给类数组对象)
const args = { 0: 'a', 1: 'b', length: 2 };
console.log(Array.prototype.slice.call(args)); // [ 'a', 'b' ]
// 现代替代:Array.from(args)

// bind 的经典用途:回调里固定 this
const counter = {
  count: 0,
  click() {
    this.count++;
    console.log('点击次数:', this.count);
  },
};
const btn = { onClick: counter.click.bind(counter) };
btn.onClick();           // 点击次数:1 ← bind 保住了 this
```

⚠ 常见误区 4.2:回调里 this 神秘丢失

setTimeout(user.hello, 100) 里 this 不是 user——回调是独立调用,this 走默认绑定。修复三选一:① user.hello.bind(user);② 箭头函数 setTimeout(() => user.hello(), 100);③ 在对象方法内部先把 this 存成变量(老代码 const self = this)。生产代码里箭头函数是最常见解法。

§4.6 箭头函数:没有自己的 this 与 arguments

箭头函数是语法糖式的新写法,但有三个与普通函数的 **本质区别**:① 没有自己的 this(取词法外层作用域的 this);② 没有 arguments;③ 不能当构造函数(没有 prototype 可挂?有 prototype 属性,但不能 new)。生产铁律:回调一律箭头函数,方法定义看情况。

```

const team = {
  members: ['李雷', '韩梅梅'],
  name: '先锋队',
  // ✅ 箭头函数回调:this 是词法外层 = team
  greet() {
    return this.members.map((m) => `${this.name}:${m}`);
  },
  // ❌ 普通函数回调:this 会丢(在回调内 this 是 undefined 或全局)
  badGreet() {
    return this.members.map(function (m) {
      return `${this.name}:${m}`; // this 不是 team!
    });
  },
};

console.log(team.greet()); // [ '先锋队:李雷', '先锋队:韩梅梅' ]
// console.log(team.badGreet()); // 报错:this.name 读 undefined

// 箭头函数没有 arguments
function normal() { return arguments.length; }
console.log(normal(1, 2, 3)); // 3
const arrow = () => arguments; // 引用外层作用域的 arguments(模块层无则报错)
// 生产:箭头函数取参一律用 rest:const arrow = (...args) => args.length

// 箭头函数不能 new
// new (() => {})(1); // ❌ TypeError: not a constructor

// 箭头函数只有"表达体"与"块体"两种形态
const square = (n) => n * n; // 表达体:直接返回
const sq2 = (n) => ({ value: n * n }); // 返回对象要加括号
const cube = (n) => { // 块体:必须写 return
  const c = n * n * n;
  return c;
};
console.log(square(4), sq2(4).value, cube(3)); // 16 16 27

```

★ 重点结论:箭头函数 this = 定义处的 this(词法绑定)

箭头函数 没有自己的 this,调用时它向上层作用域找 this。所以"在对象方法内用箭头回调"安全,"在箭头函数里再用箭头"始终是模块层的 this。判断箭头函数 this 的唯一方法:看它 定义位置 在哪个函数/对象方法里。

§4.7 闭包:函数记住定义时的环境

C 的函数指针只携带"代码地址",不携带环境;JS 的闭包同时携带 定义时可见的变量环境。函数定义后,即使外层函数已经返回,它仍能读写外层变量。生产用途:私有状态、工厂函数、计数器、防抖节流、柯里化。

```

// 经典闭包:计数器(类比 C 的 static 局部变量,但每个实例独立)
function makeCounter() {
  let count = 0;           // 被内部函数捕获
  return {
    inc: () => ++count,
    get: () => count,
    reset: () => { count = 0; },
  };
}
const c1 = makeCounter();
const c2 = makeCounter();
c1.inc(); c1.inc();
console.log(c1.get());      // 2
console.log(c2.get());      // 0 ← 两个实例互不干扰

// 私有状态:闭包模拟"私有成员"(第6章还有 # 私有字段)
function createAccount(initial) {
  let balance = initial;    // 外部无法直接访问
  return {
    deposit: (n) => { balance += n; },
    withdraw: (n) => {
      if (n > balance) return false;
      balance -= n;
      return true;
    },
    balanceOf: () => balance,
  };
}
const acc = createAccount(100);
acc.deposit(50);
console.log(acc.balanceOf()); // 150
console.log(acc.balance);     // undefined ← 私有,拿不到

// 工厂函数:按参数生成专用函数
function multiplier(factor) {
  return (n) => n * factor;   // factor 被捕获
}
const double = multiplier(2);
const triple = multiplier(3);
console.log(double(10), triple(10)); // 20 30

// 闭包陷阱:每次调用都新建环境
function loopCapture() {
  const fns = [];
  for (let i = 0; i < 3; i++) {
    fns.push(() => i);        // let:每轮新绑定
  }
  return fns;
}
console.log(loopCapture().map((f) => f())); // [ 0, 1, 2 ]

```

⚠ 常见误区 4.3:闭包里的循环变量读到最终值

这是第3章 var bug 的延续:若用 var 声明,所有闭包共享同一个绑定,读到最终值;用 let 则每轮独立。生产里在循环内创建回调(事件监听、定时器、Promise)时,一律确认循环变量是 let,或显式传参给函数。

§4.8 高阶函数:函数处理函数

把函数当参数或返回值的函数叫高阶函数。C 的 qsort 比较器是最早的高阶函数;JS 把这一思想铺满标准库。数组三件套 map/filter/reduce 必须形成肌肉记忆:

```
const prices = [199, 299, 89, 459];

// map:一一映射(数量不变,值变换)
const taxed = prices.map((p) => Math.round(p * 1.1));
console.log(taxed);           // [ 219, 329, 98, 505 ]

// filter:筛选(数量可能变)
const cheap = prices.filter((p) => p < 300);
console.log(cheap);           // [ 199, 299, 89 ] ← 299 也小于 300,保留

// reduce:聚合(变一个值)
const total = prices.reduce((acc, p) => acc + p, 0);
console.log(total);           // 1046

// 链式调用:读法从前往后
const expensiveTotal = prices
  .filter((p) => p > 200)
  .map((p) => p * 0.9)          // 打九折
  .reduce((acc, p) => acc + p, 0);
console.log(Math.round(expensiveTotal)); // 682

// 回调地狱的现代解法(第9章):回调 → Promise
function fetchUser(id, cb) {
  setTimeout(() => cb({ id, name: `用户${id}` }), 10);
}
fetchUser(7, (u) => console.log(u.name)); // 用户7

// 函数返回函数:currying(柯里化)
const addCurried = (a) => (b) => a + b;
console.log(addCurried(3)(4)); // 7
```

🚀 工程建议 4.2:用纯函数,少用副作用

纯函数(同样的输入必有同样的输出、不修改外部)是 JS 生产代码的第一原则:可测试、可缓存、可并行。map/filter/reduce 天然纯;for 循环改外部变量是副作用。团队代码评审时,优先把"循环 + push"重写为 filter/map 组合,可读性立竿见影。

§4.9 递归、入口与程序组织

```
// 递归:与 C 相同,注意 V8 对深递归会栈溢出
function fact(n) {
  return n <= 1 ? 1 : n * fact(n - 1);
}
console.log(fact(10));           // 3628800

// 深递归防护:1e5 层会 RangeError,生产大递归改迭代或拆任务
function sumTo(n) {
  let s = 0;
  for (let i = 1; i <= n; i++) s += i;
  return s;
}

// 入口:没有 main。Node 里常用模块级 main() 组织(第13章讲模块)
function main() {
  const argv = process.argv.slice(2);  // C 的 argc/argv 对应物
  console.log('参数:', argv);
}
main();
```

★ 重点结论:JS 的"参数"入口对照

C 的 `int main(int argc, char* argv[])` → Node 的 `process.argv` (`argv[0]` 是 node 路径,`argv[1]` 是脚本路径,`argc` 即 `argv.length`);浏览器程序没有命令行参数,入口是"事件"(页面加载、点击)。

本章速查表

主题	速查要点
三种定义	function 声明(提升)、const 表达式、箭头函数
传参语义	原始类型传值改不动外部;对象传引用副本,改属性影响外部
默认参数	形参表直接给;传 undefined 触发,null 不触发
变参	...rest 收集数组;arguments 是老写法,箭头函数没有
多返回值	返回对象/数组 + 解构接收
this 规则一	独立调用:undefined(严格模式)/全局对象
this 规则二	obj.method():this 指向 obj(方法拿出来调用会丢)
this 规则三	call/apply(立即调用)/bind(返回新函数)
this 规则四	new Foo():this 指向新对象;优先级 new > 显式 > 隐式 > 默认
箭头函数	无自己的 this/arguments,不能 new;this 取定义处
闭包	函数 + 定义时环境;计数器/私有状态/工厂
高阶函数	map 变换、filter 筛选、reduce 聚合、链式调用
柯里化	(a) => (b) => 结果,逐层传参
递归	与 C 相同;超深递归会 RangeError,改迭代
入口	Node:process.argv;浏览器:事件驱动,无 main

最小必会清单

- 能写出函数声明/表达式/箭头函数三种形态,并说出提升差异
- 能解释"原始类型传值、对象传引用副本"并给出改外部对象的代码
- 能写出默认参数与 rest 参数的函数,并说出 undefined 与 null 触发差异
- 能背诵 this 四规则与优先级,并各写一个 5 行示例
- 能说出 call/apply/bind 的区别,并用 bind 修复回调 this 丢失
- 能说出箭头函数与普通函数在 this/arguments/new 三个维度的差异
- 能写出闭包计数器与私有状态(账户余额)两个例子
- 能用 map/filter/reduce 链式完成"筛选+变换+聚合"
- 能解释"方法拿出来单独调用 this 丢失"并给出两种修复
- 能说出 Node 的 process.argv 与 C 的 argv 对应关系

自测题

Q 1. 写出输出:const obj = { v: 1, get() { return this.v; } }; const f = obj.get; console.log(f())

✓ 参考答案

抛 `TypeError`(严格模式下 `this` 为 `undefined`,读 `this.v` 崩溃);非严格模式返回 `undefined`。方法被拿出来独立调用,隐式绑定丢失,`this` 走默认绑定。

Q 2. 如何让 `f()` 在上题中输出 1?给出三种写法。

✓ 参考答案

① `const f = obj.get.bind(obj)`;② `const f = () => obj.get()`;③ 调用时显式绑定 `obj.get.call(obj)`。

Q 3. 写出输出:`function f(a = 1) { return a; } console.log(f(undefined), f(null), f(0))`

✓ 参考答案

1、`null`、`0`。默认参数只在参数为 `undefined` 时触发;`null` 和 `0` 都原样传入。

Q 4. 简述闭包是什么,并写出一个每次调用加 1 的计数器工厂函数。

✓ 参考答案

闭包 = 函数 + 其定义时的词法环境,内部函数可访问外层局部变量。工厂: `function makeCounter() { let n = 0; return () => ++n; }`。

Q 5. 用 `map/filter/reduce` 一行链式求出 `[1..10]` 中偶数的平方和。

✓ 参考答案

`[1,2,3,4,5,6,7,8,9,10].filter(n => n % 2 === 0).map(n => n * n).reduce((a, b) => a + b, 0)` 得 220。

Q 6. 下列代码中 `this` 分别是谁?① `p.hi()`; ② `hi.call(q)`; ③ `new Foo()`; ④ `foo()`(独立调用)

✓ 参考答案

① `p`(隐式绑定);② `q`(显式绑定);③ 新对象(`new` 绑定);④ `undefined`(默认绑定,严格模式)。

章末实验题:可配置的统计处理管线

实验 4:可配置的统计处理管线

实验目标:实现一个函数式风格的数据统计管线:数据经"过滤 → 变换 → 聚合"三步流水线处理,全面覆盖本章知识点。

实验要求:

- 任务 1(覆盖:函数定义三形态、默认参数、rest):管线工厂 `pipe(...fns)` 用 `rest` 收集处理函数,默认参数提供空管线
- 任务 2(覆盖:高阶函数、回调、纯函数):实现 `clamp`、`normalize`、`filterOutlier` 等纯函数处理器,均以"数组进、数组出"
- 任务 3(覆盖:闭包、工厂函数):实现 `makeThreshold(limit)` 工厂,返回"过滤大于 `limit` 的数据"的处理器(捕获 `limit`)
- 任务 4(覆盖:`this` 绑定、`call`/`apply`/`bind`、箭头函数):管线内用箭头函数回调;演示 `bind` 给处理器绑定配置对象
- 任务 5(覆盖:`map`/`filter`/`reduce`、多返回值解构):聚合阶段用 `reduce` 返回 `{sum, avg, max, min}` 并解构打印
- 任务 6(覆盖:参数传递语义):演示对象作为参数被处理器原地修改的"共享陷阱",并用不可变风格返回新数组规避
- 任务 7(覆盖:柯里化、递归):附加实现 `addCurried` 柯里化函数与递归 `gcd`

实验环境:Node.js 20+,运行 `node pipeline.js`。

第一步 写纯处理器:每个处理器是"数组 → 数组"的纯函数(不改入参)

第二步 写 pipe:`reduce` 依次应用,最后一个处理器可返回任意类型

第三步 后写演示:构造 20 个含噪声的分数,经"过滤异常 → 阈值截断 → 聚合"输出报告

✅ 参考答案要点

核心:pipe 高阶组合器 + 纯函数处理器 + 闭包工厂 + reduce 聚合。约 100 行,覆盖全章知识点。

```
// pipeline.js — Node.js 20+ 运行:node pipeline.js
'use strict';

// 1. 管线组合器(覆盖:rest 参数、reduce、箭头函数、高阶函数)
function pipe(...stages) {
  // 数据进入后依次经过每个 stage
  return (data) => stages.reduce((acc, stage) => stage(acc), data);
}

// 2. 纯函数处理器(覆盖:纯函数、filter/map)
const dropOutlier = (arr) => {
  if (arr.length === 0) return arr;
  const avg = arr.reduce((a, b) => a + b, 0) / arr.length;
  return arr.filter((v) => Math.abs(v - avg) <= avg * 0.5); // 剔除偏离均值 50% 的
};

const clamp100 = (arr) => arr.map((v) => Math.min(100, Math.max(0, v)));
const round2 = (arr) => arr.map((v) => Math.round(v * 100) / 100);

// 3. 闭包工厂(覆盖:闭包、工厂函数、捕获变量)
function makeThreshold(limit) {
  return (arr) => arr.filter((v) => v <= limit); // limit 被闭包捕获
}
const under90 = makeThreshold(90);

// 4. 聚合器(覆盖:reduce、多返回值对象 + 解构)
function aggregate(arr) {
  if (arr.length === 0) return { sum: 0, avg: 0, max: 0, min: 0, count: 0 };
  const sum = arr.reduce((a, b) => a + b, 0);
  return {
    sum,
    avg: sum / arr.length,
    max: Math.max(...arr),
    min: Math.min(...arr),
    count: arr.length,
  };
}

// 5. 参数传递演示(覆盖:对象共享陷阱 vs 不可变风格)
function sharedBug(scores) {
  scores.sort((a, b) => a - b); // 原地排序,改了外部!
  return scores;
}
function safeSorted(scores) {
  return [...scores].sort((a, b) => a - b); // 复制后排序,外部不受影响
}

// 6. this 绑定演示(覆盖:bind、箭头函数)
const config = { prefix: '[分数]' };
function report(label, data) {
  return `${this.prefix}${label}:${data}`; // 依赖 this.prefix
}
const boundReport = report.bind(config);

// 7. 柯里化与递归(覆盖:柯里化、递归)
```

```

const addCurried = (a) => (b) => a + b;
function gcd(a, b) {
  return b === 0 ? Math.abs(a) : gcd(b, a % b);
}

// 8. 主流程(覆盖:函数表达式、main 组织、模板字符串)
function main() {
  const raw = [];
  for (let i = 0; i < 20; i++) {
    raw.push(Math.round(40 + Math.random() * 80)); // 40~120 含噪声
  }

  // 组合管线:剔除异常 → 截断 100 → 舍入 → 过滤 > 90 → 聚合
  const pipeline = pipe(dropOutlier, clamp100, round2, under90, aggregate);
  const stats = pipeline(raw);

  console.log('原始数据:', raw.join(' '));
  console.log(boundReport('管线结果', JSON.stringify(stats)));

  // 共享陷阱演示
  const original = [3, 1, 2];
  sharedBug(original);
  console.log('sharedBug 后外部也被排序:', original); // [ 1, 2, 3 ]

  const fresh = [3, 1, 2];
  const sorted = safeSorted(fresh);
  console.log('safeSorted 后外部不变:', fresh, '副本:', sorted);

  // 柯里化与递归演示
  console.log('addCurried(3)(4) =', addCurried(3)(4));
  console.log('gcd(48, 36) =', gcd(48, 36));
}

main();

```

设计说明:pipe 用 reduce 实现"数据穿过每个 stage",最后一个 stage 输出任意类型(聚合器返回对象);所有处理器保持纯函数,唯一会改外部的 sharedBug 被单独隔离演示,强调"复制后修改"的不可变风格;boundReport 用 bind 固定 this,呼应 §4.5;makeThreshold 演示闭包捕获工厂参数。运行输出包含管线统计、绑定报告、共享陷阱对比、柯里化与 gcd 五部分。

下一章预告:第5章 内存管理 —— V8 的标记-清除、内存泄漏四大场景、WeakMap 与 Buffer,GC 语言里"内存问题"长什么样。